```
# Question 1
import numpy as np
import matplotlib.pyplot as plt

# Below i am setting the initial conditions
tos = 100.0
Mx = 0.5
d_x = 1.0
d_y = 1.0
d_t = 0.1
# here the range of x values and y values
x=np.arange(-100-3*d_x,100+3*d_x,d_x)
y=np.arange(-100-3*d_y,100+3*d_y,d_y)
# creating a mesh for the x & y
X,Y = np.meshgrid(x,y)
shx = np.shape(x)[0]
shy = np.shape(y)[0]
# Initializing my U vector
U = np.zeros(shape=(shx,shy,4))
# calculating Radius cosine , sine and V_theta
rad = np.sqrt(X**2 + Y**2)
rad[np.where(rad ==0)] = 1.0
costh = X/rad
sinth = Y/rad
Vth = Mx*costh + np.sqrt(1.0 - (Mx*sinth)**2)
# same cosine , sine and vth but now at each grid point 4 values
costh_big = np.zeros(shape=np.shape(U))
sinth_big = np.zeros(shape=np.shape(U))
Vth_big = np.zeros(shape=np.shape(U))
rad_big = np.zeros(shape=np.shape(U))
for i in range(4):
    costh_big[:,:,i] = costh[:,:]
    sinth_big[:,:,i] = sinth[:,:]
    Vth_big[:,:,i] = Vth[:,:]
    rad_big[:,:,i] = rad[:,:]

# All the respective stencil coefficients

# central stencil
a0 = 0.0
a1 = 0.77088238051822552
a2  = -0.166705904414580469
a3  = 0.02084314277031176
a_1 = -a1
a_2 = -a2
a_3 = -a3
cent = np.array([a_3,a_2,a_1,a0,a1,a2,a3])
# forward
a60_0 = -2.19228033900
a60_1 = 4.74861140100
a60_2 = -5.10885191500
a60_3 = 4.46156710400
a60_4 = -2.83349874100
a60_5 = 1.12832886100
a60_6 = -0.20387637100
fwd_3 = np.array([a60_0,a60_1,a60_2,a60_3,a60_4,a60_5,a60_6])
back_3 = -fwd_3[::-1]
#.............
a51__1 = -0.20933762200
```

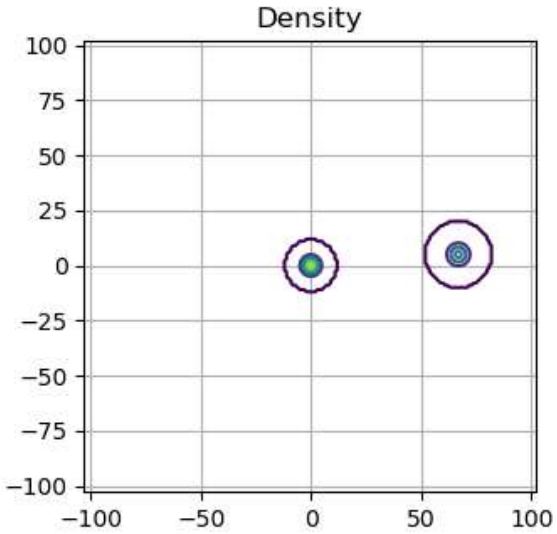
```
a51_0 = -1.08487567600
a51_1 = 2.14777605000
a51_2 = -1.38892832200
a51_3 = 0.76894976600
a51_4 = -0.28181465000
a51_5 = 0.48230454000e-1
fwd_2 = np.array([a51__1,a51_0,a51_1,a51_2,a51_3,a51_4,a51_5])
back_2 = -fwd_2[::-1]
#.....................
a42__2 = 0.49041958000e-1
a42__1 = -0.46884035700
a42_0 = -0.47476091400
a42_1 = 1.27327473700
a42_2 = -0.51848452600
a42_3 = 0.16613853300
a42_4 = -0.26369431000e-1
fwd_1 = np.array([a42__2,a42__1,a42_0,a42_1,a42_2,a42_3,a42_4])
back_1 = -fwd_1[::-1]

# time coefficient
b0 = 2.3025580888383
b1 = -2.4910075998482
b2 = 1.5743409331815
b3 = -0.3858914221716

# defining the initial condition
# function call for intital conditions
def ini_con(xa,ya,xe,ye,xv,yv):
    e_a = 0.01
    e_e = 0.005
    e_v = 0.007
    al_a = np.log(2)/9.0
    al_e = np.log(2)/16.0
    al_v = np.log(2)/25.0

    U[:,:,0] = e_a*np.exp(-al_a*((X-xa)**2+(Y-ya)**2)) +
e_e*np.exp(-al_e*((X-xe)**2+(Y-ye)**2))
    U[:,:,1] = e_v*(Y-yv)*np.exp(-al_v*((X-xv)**2+(Y-yv)**2))
    U[:,:,2] = -e_v*(X-xv)*np.exp(-al_v*((X-xv)**2+(Y-yv)**2))
    U[:,:,3] = e_a*np.exp(-al_a*((X-xa)**2+(Y-ya)**2))
    return U

# function to calculate E vector
def E_cal(Mx,U):
    E = np.zeros(shape = np.shape(U))
    E[:,:,0] = Mx*U[:,:,0] + U[:,:,1]
    E[:,:,1] = Mx*U[:,:,1] + U[:,:,3]
    E[:,:,2] = Mx*U[:,:,2]
    E[:,:,3] = Mx*U[:,:,3] + U[:,:,1]
    return E

# function to calculate F vector
def F_cal(U):
    F = np.zeros(shape = np.shape(U))
    F[:,:,[0,2,3]] = U[:,:,[2,3,2]]
    return F

# Divinding the domains for x and y values
xi_1 = np.arange(3)
```

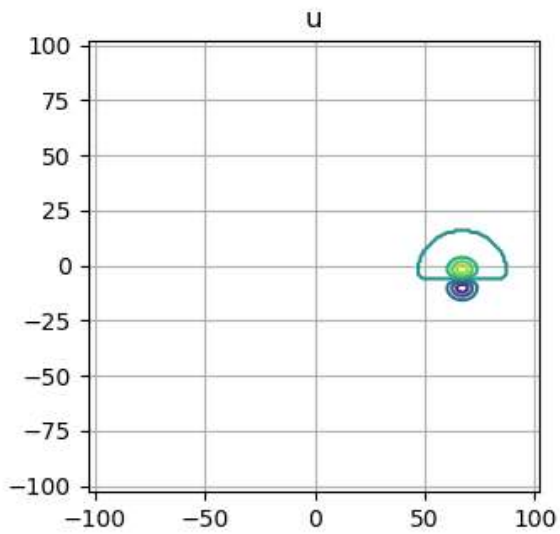
```
xi_2 = np.arange(3,shx-3)
xi_3 = np.arange(shx-3,shx)

yi_1 = np.arange(3)
yi_2 = np.arange(3,shy-3)
yi_3 = np.arange(shy-3,shy)
yi_4 = np.append(yi_1,yi_3)
yi_5 = np.arange(shy)

# making regions for computaion
# region 1 is left boundary including corners
rx_1,ry_1 = np.meshgrid(xi_1,yi_5)
# region 2 is top and bottom boundary excluding corners
rx_2,ry_2 = np.meshgrid(xi_2,yi_4)
# region 3 is right boundary including corners
rx_3,ry_3 = np.meshgrid(xi_3,yi_5)
# region 4 is region excluding boundary
rx_4,ry_4 = np.meshgrid(xi_2,yi_2)

# intializing the derivative matrix
dU_dt = np.zeros(shape=(shx,shy,4,4))

# time variable
tim = 0.0

# calculating intial U , E, F, and derivatives
U = ini_con(0.0,0.0,67.0,5.0,67.0,-6.0)
E = E_cal(Mx,U)
F = F_cal(U)
dU_dx = np.zeros(shape = np.shape(U))
dU_dy = np.zeros(shape = np.shape(U))

while tim<tos:
    # calculating d/dx derivatives at left boundary
    dU_dx[:,0,:] =
fwd_3[0]*U[:,0,:]+fwd_3[1]*U[:,1,:]+fwd_3[2]*U[:,2,:]+fwd_3[3]*U[:,3,:]+fwd_3[4]*U[:,4,:
]+fwd_3[5]*U[:,5,:]+fwd_3[6]*U[:,6,:]
    dU_dx[:,1,:] =
fwd_2[0]*U[:,0,:]+fwd_2[1]*U[:,1,:]+fwd_2[2]*U[:,2,:]+fwd_2[3]*U[:,3,:]+fwd_2[4]*U[:,4,:
]+fwd_2[5]*U[:,5,:]+fwd_2[6]*U[:,6,:]
    dU_dx[:,2,:] =
fwd_1[0]*U[:,0,:]+fwd_1[1]*U[:,1,:]+fwd_1[2]*U[:,2,:]+fwd_1[3]*U[:,3,:]+fwd_1[4]*U[:,4,:
]+fwd_1[5]*U[:,5,:]+fwd_1[6]*U[:,6,:]
    # calculating d/dx derivatives at right boundary
    dU_dx[:,shx-1,:] =
back_3[6]*U[:,shx-1,:]+back_3[5]*U[:,shx-2,:]+back_3[4]*U[:,shx-3,:]+back_3[3]*U[:,shx-4
,:]+back_3[2]*U[:,shx-5,:]+back_3[1]*U[:,shx-6,:]+back_3[0]*U[:,shx-7,:]
    dU_dx[:,shx-2,:] =
back_2[6]*U[:,shx-1,:]+back_2[5]*U[:,shx-2,:]+back_2[4]*U[:,shx-3,:]+back_2[3]*U[:,shx-4
,:]+back_2[2]*U[:,shx-5,:]+back_2[1]*U[:,shx-6,:]+back_2[0]*U[:,shx-7,:]
    dU_dx[:,shx-3,:] =
back_1[6]*U[:,shx-1,:]+back_1[5]*U[:,shx-2,:]+back_1[4]*U[:,shx-3,:]+back_1[3]*U[:,shx-4
,:]+back_1[2]*U[:,shx-5,:]+back_1[1]*U[:,shx-6,:]+back_1[0]*U[:,shx-7,:]
    # calculating d/dx derivatives at center
    dU_dx[:,xi_2,:] =
cent[0]*U[:,xi_2-3,:]+cent[1]*U[:,xi_2-2,:]+cent[2]*U[:,xi_2-1,:]+cent[3]*U[:,xi_2,:]+ce
nt[4]*U[:,xi_2+1,:]+cent[5]*U[:,xi_2+2,:]+cent[6]*U[:,xi_2+3,:]

    dU_dx = dU_dx/d_x
```

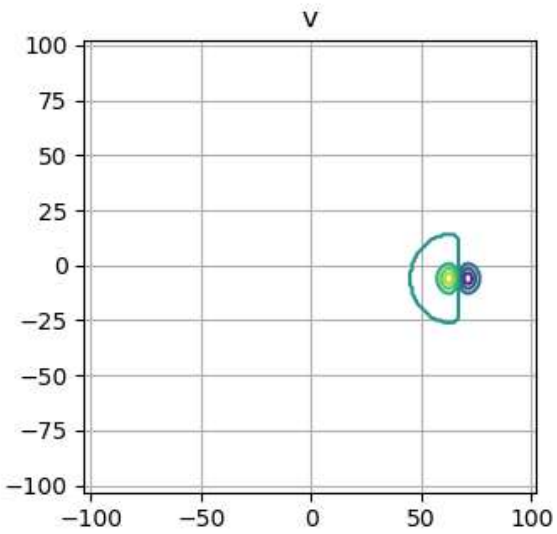
```
    # calculating d/dy derivatives at bottom boundary
    dU_dy[0,:,:] =
fwd_3[0]*U[0,:,:]+fwd_3[1]*U[1,:,:]+fwd_3[2]*U[2,:,:]+fwd_3[3]*U[3,:,:]+fwd_3[4]*U[4,:,:
]+fwd_3[5]*U[5,:,:]+fwd_3[6]*U[6,:,:]
    dU_dy[1,:,:] =
fwd_2[0]*U[0,:,:]+fwd_2[1]*U[1,:,:]+fwd_2[2]*U[2,:,:]+fwd_2[3]*U[3,:,:]+fwd_2[4]*U[4,:,:
]+fwd_2[5]*U[5,:,:]+fwd_2[6]*U[6,:,:]
    dU_dy[2,:,:] =
fwd_1[0]*U[0,:,:]+fwd_1[1]*U[1,:,:]+fwd_1[2]*U[2,:,:]+fwd_1[3]*U[3,:,:]+fwd_1[4]*U[4,:,:
]+fwd_1[5]*U[5,:,:]+fwd_1[6]*U[6,:,:]
    # calculating d/dy derivatives at top boundary
    dU_dy[shy-1,:,:] =
back_3[6]*U[shy-1,:,:]+back_3[5]*U[shy-2,:,:]+back_3[4]*U[shy-3,:,:]+back_3[3]*U[shy-4,:
,:]+back_3[2]*U[shy-5,:,:]+back_3[1]*U[shy-6,:,:]+back_3[0]*U[shy-7,:,:]
    dU_dy[shy-2,:,:] =
back_2[6]*U[shy-1,:,:]+back_2[5]*U[shy-2,:,:]+back_2[4]*U[shy-3,:,:]+back_2[3]*U[shy-4,:
,:]+back_2[2]*U[shy-5,:,:]+back_2[1]*U[shy-6,:,:]+back_2[0]*U[shy-7,:,:]
    dU_dy[shy-3,:,:] =
back_1[6]*U[shy-1,:,:]+back_1[5]*U[shy-2,:,:]+back_1[4]*U[shy-3,:,:]+back_1[3]*U[shy-4,:
,:]+back_1[2]*U[shy-5,:,:]+back_1[1]*U[shy-6,:,:]+back_1[0]*U[shy-7,:,:]
    # calculating d/dy derivatives at center
    dU_dy[yi_2,:,:] =
cent[0]*U[yi_2-3,:,:]+cent[1]*U[yi_2-2,:,:]+cent[2]*U[yi_2-1,:,:]+cent[3]*U[yi_2,:,:]+ce
nt[4]*U[yi_2+1,:,:]+cent[5]*U[yi_2+2,:,:]+cent[6]*U[yi_2+3,:,:]

    dU_dy = dU_dy/d_y

    # calculating time derivative with radiation boundary condition in region 1 and 2
    dU_dt[ry_1,rx_1,:,0] = -Vth_big[ry_1,rx_1,:]*(
costh_big[ry_1,rx_1,:]*dU_dx[ry_1,rx_1,:] +  sinth_big[ry_1,rx_1,:]*dU_dy[ry_1,rx_1,:] +
0.5*U[ry_1,rx_1,:]/rad_big[ry_1,rx_1,:] )
    dU_dt[ry_2,rx_2,:,0] = -Vth_big[ry_2,rx_2,:]*(
costh_big[ry_2,rx_2,:]*dU_dx[ry_2,rx_2,:] +  sinth_big[ry_2,rx_2,:]*dU_dy[ry_2,rx_2,:] +
0.5*U[ry_2,rx_2,:]/rad_big[ry_2,rx_2,:] )

    # calculating time derivative with outflow boundary condition in region 3
    dU_dt[ry_3,rx_3,3,0] = -Vth[ry_3,rx_3]*( costh[ry_3,rx_3]*dU_dx[ry_3,rx_3,3] +
sinth[ry_3,rx_3]*dU_dy[ry_3,rx_3,3] + 0.5*U[ry_3,rx_3,3]/rad[ry_3,rx_3] )
    dU_dt[ry_3,rx_3,0,0] = Mx*( dU_dx[ry_3,rx_3,3] - dU_dx[ry_3,rx_3,0] ) +
dU_dt[ry_3,rx_3,3,0]
    dU_dt[ry_3,rx_3,1,0] = -( dU_dx[ry_3,rx_3,3] + Mx*dU_dx[ry_3,rx_3,1])
    dU_dt[ry_3,rx_3,2,0] = -( dU_dy[ry_3,rx_3,3] + Mx*dU_dx[ry_3,rx_3,2])

    # calculating dE/dx in region 4
    dE_dx =  (cent[3]*E[ry_4,rx_4,:] + cent[4]*E[ry_4,rx_4+1,:] +
cent[5]*E[ry_4,rx_4+2,:] + cent[6]*E[ry_4,rx_4+3,:])
    dE_dx += (cent[2]*E[ry_4,rx_4-1,:] + cent[1]*E[ry_4,rx_4-2,:] +
cent[0]*E[ry_4,rx_4-3,:])
    dE_dx *= (1/d_x)

    # calculating dF/dy in region 4
    dF_dy =  (cent[3]*F[ry_4,rx_4,:] + cent[4]*F[ry_4+1,rx_4,:] +
cent[5]*F[ry_4+2,rx_4,:] + cent[6]*F[ry_4+3,rx_4,:])
    dF_dy += (cent[2]*F[ry_4-1,rx_4,:] + cent[1]*F[ry_4-2,rx_4,:] +
cent[0]*F[ry_4-3,rx_4,:])
    dF_dy *= (1/d_y)

    # calculating time derivative in region 4 by defination
```

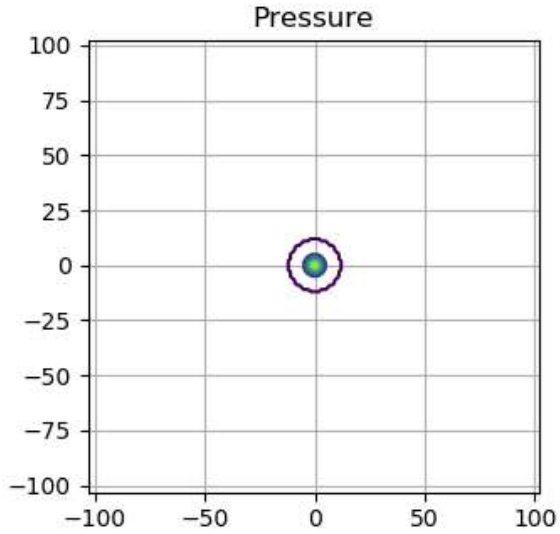
```
    dU_dt[ry_4,rx_4,:,0] = - dE_dx - dF_dy

    # Updating U
    U = U + d_t * (b0*dU_dt[:,:,:,0] + b1*dU_dt[:,:,:,1] + b2*dU_dt[:,:,:,2] +
b3*dU_dt[:,:,:,3])

    # book keeping of time derivative at lower three time levels
    dU_dt[:,:,:,1:4] = dU_dt[:,:,:,0:3]

    # calculation of updated E and F
    E = E_cal(Mx,U)
    F = F_cal(U)

    # time increasing
    tim += d_t

# rounding off U for clear contour plots
U = np.round(U,6)
# plotting
fig,ax =  plt.subplots(2,2)
fig.suptitle('Plots at time = ' +str(tos)+ ' sec')
ax[0][0].contour(X,Y,U[:,:,0])
ax[0][0].set_title('Density')
ax[0][0].set_aspect('equal')
ax[0][0].grid(True)

ax[0][1].contour(X,Y,U[:,:,1])
ax[0][1].set_title('u')
ax[0][1].set_aspect('equal')
ax[0][1].grid(True)

ax[1][0].contour(X,Y,U[:,:,2])
ax[1][0].set_title('v')
ax[1][0].set_aspect('equal')
ax[1][0].grid(True)

ax[1][1].contour(X,Y,U[:,:,3])
ax[1][1].set_title('Pressure')
ax[1][1].set_aspect('equal')
ax[1][1].grid(True)

plt.show()
```

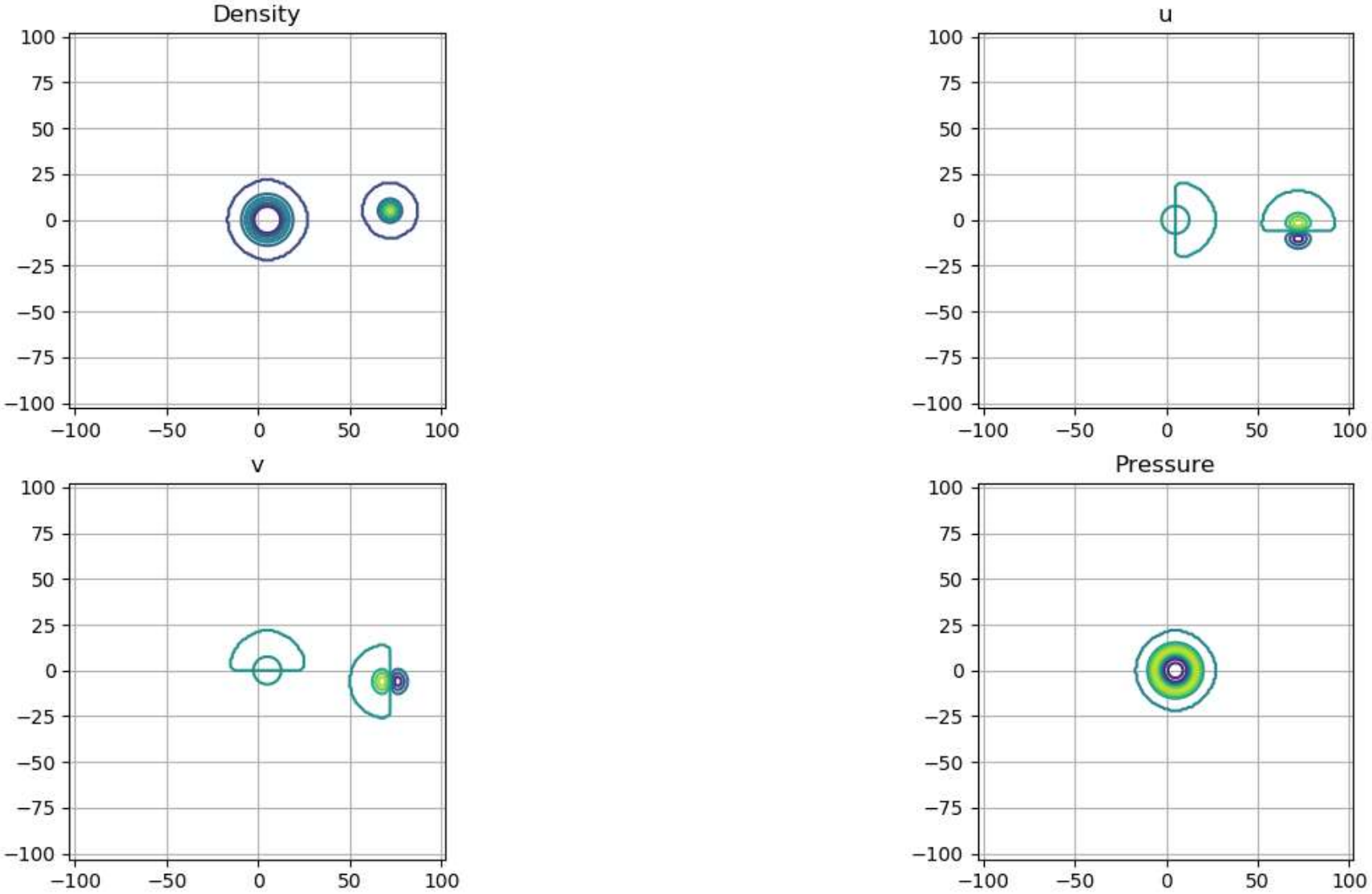
Plots at time = 0 sec



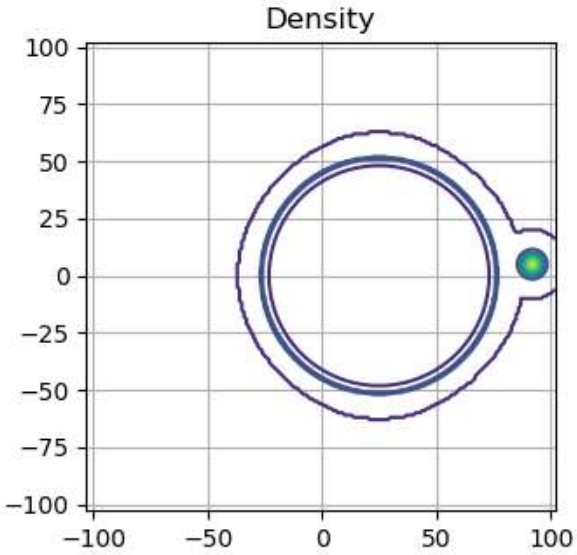


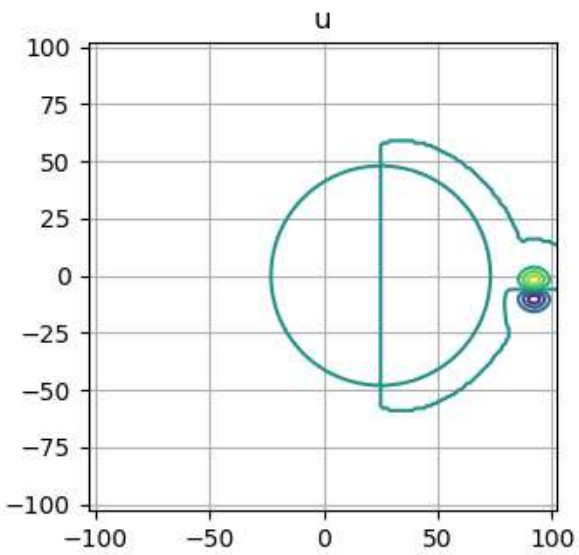


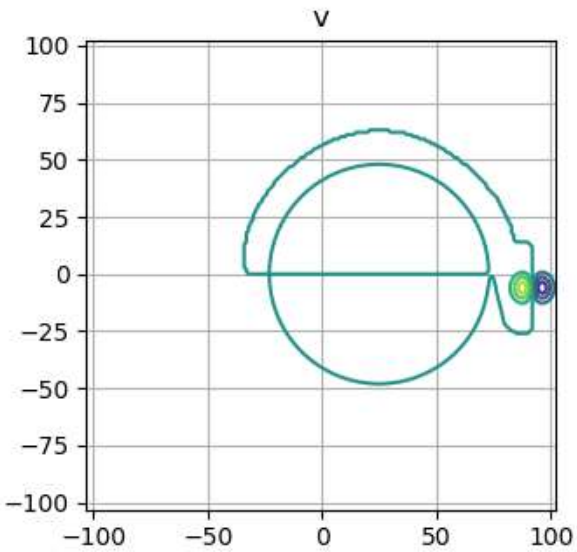


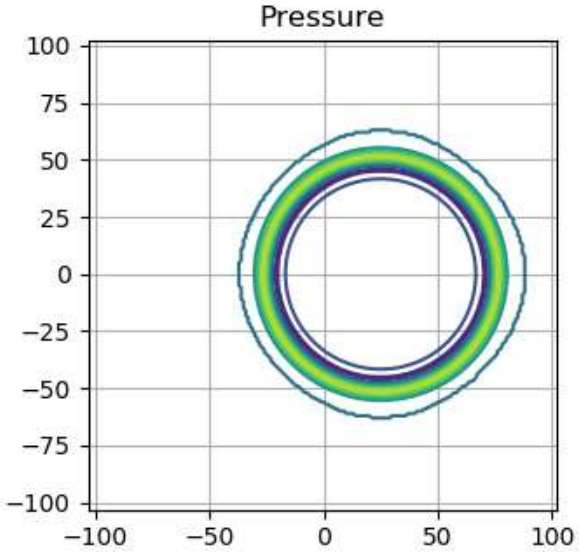

Plots at time = 10.0 sec
Density
u
v
Pressure
100
75
50
25
0
−25
−50
−75
−100
−100
−50
0
50
100

Plots at time = 50.0 sec









Plots at time = 100.0 sec